Homework Assignment 1

The first homework assignment is due on **September 27th at 11:59PM**. Please closely follow instructions listed in the syllabus.

Note: Use `print()` function to display output results after each part of homework problems. For example, `print(vec1)`.

Problem 1 [7pt]

1. Create vectors containing {2.5, 4.5, 6.5, 8.5, 10.5, 12.5, 14.5} elements in three different ways, using the following functions: `c()`, `seq()` with the `by` argument, and `seq()` with the `length.out` argument. Name them `vector1`, `vector2`, and `vector3`, respectively.

2. Append four additional elements to `vector2`: 10, 20, 30, and 40. Ensure that 10 becomes the second element in the updated `vector2`, 20 is the first element considering that 10 has been already added, 30 occupies the 7th position given the presence of 10 and 20, and 40 is positioned as the last element after including all the other numbers.

3. Form a new vector encompassing {2, 3, 6, 10, 15, 18, 22, 25, 27, 30, 31, 35, 42} elements and label it `vector4`. Extract elements greater than 15 and not divisible by 2 through a logical statement, generating a new vector named `vector5`. Verify the presence of the number 27 within `vector5` using a logical operator.

Problem 2 [9pt]

1. Create a list `listX` with the following elements:

- A character string "Data Science"
- A numeric value 100

- A vector {-3, 0, 3, 6}
- A logical value `TRUE`
- A sublist containing vectors {“A”, “B”, “C”} and {2.5, 5.5, 7.5}

2. Extract the third element of `listX` and assign it to `vector6`.

3. Convert `listX` into a single vector using `unlist()`. Explain why the resulting structure appears as it does.

4. Modify `listX` by replacing the second element with a new value: 200.

5. Create a new list `listY` by merging `listX` with another list containing “Statistics”, 50, and `FALSE`. Compare the structures of `listX` and `listY`.

Problem 3 [9pt]

1. Create a data frame `dfA` with the following variables:

- `Team` - {“Alpha”, “Beta”, “Gamma”, “Delta”, “Epsilon”, “Beta”, “Alpha”, “Gamma”, “Delta”, “Epsilon”}
- `Role` - {“Analyst”, “Manager”, “Developer”, “Analyst”, “Manager”, “Developer”, “Analyst”, “Manager”, “Developer”, “Analyst”}
- `Score` - {82, 90, 75, 88, 79, 95, 80, 85, 76, 89}
- `Certified` - {“Yes”, “No”, “Yes”, “No”, “Yes”, “Yes”, “No”, “Yes”, “No”, “Yes”}
- `Experience` - {3, 5, 7, 2, 6, 8, 4, 3, 5, 9}

2. Compute summary statistics for the `Score` variable.

3. Convert `Certified` into a factor if it is not already. Display the frequency table of its values.

4. Add a new column `High_Performer`, which is `TRUE` if `Score` is greater than 85 and `Experience` is more than 4, otherwise `FALSE`.

5. Extract all employees who belong to the `Gamma` team and are certified.

Problem 4 [9pt]

1. Generate a matrix named `matrix1` with 5 columns, filled row-wise with the elements: 10, 3, 6, 23, -5, -4, 13, 17, 5, 6, -7, -10, 13, 39, 20, 2, 1, 9, 11, -22, 23, -15, -3, 6, 12.

2. Employ a `For loop` to update the elements of `matrix1` according to the following criteria: replace negative values with 0, for elements greater than or equal to 0 but less than 5, replace them with 5; for elements less than 15 but greater than 10, replace them with 10; for elements that do not meet the specified conditions, substitute them with 100.

3. Initiate an empty vector `vector7`. Utilize a `while loop` to fill `vector7` with odd numbers greater than 0 and less than 30, excluding those odd numbers divisible by 3.

Problem 5 [9pt]

1. Write a function `categorize_numbers()` that takes a numeric vector as input and classifies each element into one of three categories:

- `Small` if the number is less than 10
- `Medium` if the number is between 10 and 20 (inclusive)
- `Large` if the number is greater than 20

The function should return a character vector with the corresponding category labels. Test your function with the vector: {4, 10, 15, 22, 8, 18, 27, 5, 12}.

2. Create a function `matrix_summary()` that accepts a numeric matrix and returns a list containing the following elements:

- `Row Sums` - a vector with the sum of each row
- `Column Means` - a vector with the mean of each column
- `Max Value` - the maximum value in the entire matrix
- `Min Value` - the minimum value in the entire matrix

Hint: `sum(), mean(), max(), min()` functions will come in-handy.

Test your function on the matrix:

$$\begin{bmatrix} 5 & 12 & 18 \\ 7 & 14 & 22 \\ 9 & 10 & 25 \\ 6 & 17 & 30 \end{bmatrix}$$